



1. Investiga qué versión del kernel tienes, añadiendo una captura que la muestre en el reporte

El Kernel que tengo se puede ver en la captura correspondiente:

```
mariososa@vbox:~/Documents$ cd kernels/  
mariososa@vbox:~/Documents/kernels$ ls  
linux-6.14.tar.xz  
mariososa@vbox:~/Documents/kernels$ uname -r  
6.13.10-200.fc41.x86_64  
mariososa@vbox:~/Documents/kernels$
```

Figure 1: uname del sistema

Se hacen algunos pasos adicionales que se ven algo así:

```
21 tar -xvf linux-6.14.tar.xz -C linux-6.14  
22 sudo dnf group install "c-development"  
23 sudo dnf install ncurses-devel  
24 sudo dnf install qwt-qt6-devel  
25 sudo dnf install gtk+-devel gtk2-devel libglade2-devel  
26 sudo dnf install elfutils-libelf-devel  
27 sudo dnf install elfutils-libelf-devel  
28 sudo dnf install openssl-devel  
29 sudo dnf install openssl  
30 grep -R "Linux" include/linux
```

Figure 2: pasos extra

Para cambiar el nombre del nuevo kernel lo hice medio manualmente, use estos comandos y dentro del archivo cambie donde dice el UTS SYSNAME:

```

54  ls
55  grep -R "Linux" include/linux
56  cd linux-6.14/
57  vim include/linux/uts.h
58  ls

```

Figure 3: Cambiandole el nombre al kernel uwu

Cabe destacar que cuando estaba haciendo la compilación del kernel usando make se tardo un monton y ademas lo tuve que reiniciar varias veces porque gnome lo detenia por usar demasiados recursos. Ademas le tuve que dar mas cores y mas memoria ram ademas de un HDD mas grande.

Recomiendo si lo van a hacer usen al menos 30gb de HDD y 4gb de RAM.

Despues de horas de intentarlo y que regresara errores a la media hora, logre hacer la compilación, aqui una imagen de despues cuando ya complete todos los pasos instalando y otra ya con el nombre cambiando del kernel:

```

mariososa@vbox:~/Documents/kernels/linux-6.14$ make -j$(nproc)
DESCEND objtool
INSTALL libsubcmd_headers
CALL     scripts/checksyscalls.sh
CHK      kernel/kheaders_data.tar.xz
TEST     posttest
arch/x86/tools/insn_decoder_test: success: Decoded and checked 7416971 instructions
TEST     posttest
arch/x86/tools/insn_sanity: Success: decoded and checked 1000000 random instructions with 0 errors (seed:0x527cb9be)
Kernel: arch/x86/boot/bzImage is ready

```

Figure 4: Compilado

```

mariososa@vbox:~/Documents/kernels/linux-6.14$ uname -a
JuanMarioSosaRomo vbox 6.14.0 #1 SMP PREEMPT_DYNAMIC Sun Apr 13 07:27:58 CST 2025 x86_64 GNU/Linux
mariososa@vbox:~/Documents/kernels/linux-6.14$

```

Figure 5: Final

Preguntas

¿Qué es el bootloader?

El *bootloader* es un programa que se ejecuta justo despues del UEFI o BIOS, se busca en su lugar pre definido en memoria y cabe destacar es un lugar bastante pequeño comparado al kernel desde donde se dedicara a hacer principalmente configuraciones e inicializar el Kernel y el SO, de manera resumida, ajusta el estado del HW para que sea compatible con el Kernel y todo funcione bien, despues de esto el Kenel es quien toma el control.

¿Qué son los kernel modules?

Los *kernel modules* son pedazos de codigo que pueden ser cargados y descargados del nucleo sin necesidad de reiniciar el sistema, con ellos podemos personalizar y ampliar las capacidades del kernel de manera dinamica haciendo nuestro sistema flexible y adaptable.

Los mas comunes de estos son los *drivers* que nos ayudan a gestionar y comunicarnos con el hardware, aunque tambien existen por ejemplo para almacenamiento externo como usbs o red.